

COMP202 Week 5 - solutions

1. (a) Solve the following instance of the $\{0,1\}$ Knapsack Problem with four items where the maximum allowed weight is $W_{\max} = 10$.

i	1	2	3	4
b_i	25	15	20	36
w_i	7	2	3	6

Solution: We proceed with this solution as outlined in the notes. We define $B[k, w]$ to be the optimal solution that can be obtained using only the first k items, with maximum allowed total weight of w . Here k ranges from 1 to 4, while w ranges from 0 to 10. We also define $B[0, w] = 0$ for all w .

Then, as we did in class, we have to fill in the table, one row at a time, to find the optimal solution, using the recurrence type of equation that can be found in the class notes. This table will end up having these values:

	0	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	0	25	25	25	25
2	0	0	15	15	15	15	15	25	25	40	40
3	0	0	15	20	20	35	35	35	35	40	45
4	0	0	15	20	20	35	36	36	51	56	56

So the maximum benefit obtainable is 56. Since this example is small, we can easily figure out that we want to take items 3 and 4 to obtain this benefit. Otherwise, as also mentioned in class, we can “trace back” through the table to find this out.

- (b) For comparison, what is the solution to the corresponding Fractional Knapsack problem with the same value of $W_{\max}$ (use the greedy method we discussed for this “Fractional” version)?

Solution: Recall that we compute the value indices and take items in order of the value indices to fill up the knapsack. So I have added another line to the table below for the value indices.

i	1	2	3	4
b_i	25	15	20	36
w_i	7	2	3	6
v_i	$3\frac{4}{7}$	$7\frac{1}{2}$	$6\frac{2}{3}$	6

So we select items in the order 2,3,4,1.

Doing so will lead us to the Fractional Knapsack solution consisting of items 2, 3, and $\frac{5}{6}$ of item 4, for a total benefit of $15 + 20 + \frac{5}{6}(36) = 65$.

2. Do the same as above (find solutions to both the $\{0,1\}$ and Fractional Knapsack problems) for this collection of items, where $W_{\max} = 11$.

i	1	2	3	4	5
b_i	14	12	15	20	16
w_i	4	3	8	7	3

Solution: Here's the table you would get for the $\{0,1\}$ version of this

problem:		1	2	3	4	5	6	7	8	9	10	11
	1	0	0	0	14	14	14	14	14	14	14	14
	2	0	0	12	14	14	14	26	26	26	26	26
	3	0	0	12	14	14	14	26	26	26	26	27
	4	0	0	12	14	14	14	26	26	26	32	34
	5	0	0	16	16	16	28	30	30	30	42	42

From this table, you can see that the maximum benefit for the $\{0,1\}$ Knapsack Problem for this set of items is 42. What is the set of items that achieves this benefit? You can find that by tracing backwards through the table. Since $B[5, 11] > B[4, 11]$ we see that item 5 must be used in the solution.

Then we want to compare $B[4, 8]$ and $B[3, 8]$. Since they are the same, we conclude that item 4 is not used in the best solution. Continuing in this way, we can determine that the best solution uses items 5, 2, and 1.

For the Fractional case, we again compute the value indices.

i	1	2	3	4	5
b_i	14	12	15	20	16
w_i	4	3	8	7	3
v_i	$3\frac{2}{3}$	4	$1\frac{7}{8}$	$2\frac{6}{7}$	$5\frac{1}{3}$

So we select items (greedily) in the order 5,2,1,4,3.

This will give us the greedy solution for the Fractional case consisting of items 5,2,1 and $\frac{1}{7}$ of item 4, for a total benefit of $42 + \frac{1}{7}(20) = 44\frac{6}{7}$.

3. (a) Describe an efficient greedy algorithm for making change for a specified value using a minimum number of coins, assuming that there are four denominations of currency of coins with values 27, 9, 3, 1.

Argue why your algorithm is correct.

- (b) Consider a post office that sells stamps in three different denominations, 1p, 7p, and 10p. Show that the natural greedy algorithm (hopefully the one you designed as a solution to the previous question) won't necessarily give a right answer by exhibiting an input where it goes wrong. Design a dynamic programming algorithm that will find the minimum number of stamps necessary for a postage value of N pence.

What is the running time of your algorithm?

Solution:

- (a) The algorithm to use here is the obvious one: Given a target value N , take as many 27s as possible until you can take no more, then take as many 9s, etc. So the algorithm is "greedy" in the sense that you take as large a coin as possible until you can't take any more,

then you move onto the next largest coin. This will eventually give you a number of coins that will add up to the given value N .

Why is this correct, in the sense that it will give you the minimum number of coins whose value sums to the given value in question?

An obvious first observation is that for any value N , an optimal solution will use at most 3 1s, at most 3 3s, and at most 3 9s for a set of coins whose total is N . This is because otherwise we could replace a set of 3 coins by a single larger coin to obtain a better solution.

This is essentially the argument why the greedy algorithm given above will give you the smallest number of coins. The greedy procedure will start with a value N , and take as many 27s as possible until the resulting value is less than or equal to 26. Then you move onto the 9s, until the remaining value is less than or equal to 8, etc. Using these observations above, the argument can be made more precise and mathematically formal to show that this greedy algorithm will minimize the number of coins used.

- (b) Here I give quite a detailed solution to this problem. Note that I don't necessarily expect you to provide pseudo-code for your proposed solution. Your description should be sufficiently detailed to give some argument as to why your solution is correct.

Let $S[n]$ denote the minimum number of stamps needed to make postage for n pence. We clearly then have

$$\begin{aligned} S[0] &= 0, S[1] = 1, S[2] = 2, S[3] = 3, S[4] = 4, S[5] = 5, \\ S[6] &= 6, S[7] = 1, S[8] = 2, S[9] = 3, S[10] = 1. \end{aligned}$$

These cases above should be considered our "base cases" and we then work from these to compute values of $S[n]$ for higher values of n .

The main idea, then is to consider what happens if we use a stamp of a particular value. For example, if we want postage of n pence, then we can clearly get it by taking one stamp of 1p, and $S[n-1]$ stamps (of appropriate values) to make up the remaining postage of $(n-1)$ p. Or if we take a 7p stamp, then we need $S[n-7]$ stamps (of the right values) to make up the rest, and similarly if we take a 10p stamp. This idea is essentially the heart of the dynamic programming algorithm that we use. The pseudo-code is given below:

STAMPS(n)

```

▷ We assume here that  $n \geq 0$ .
▷ This procedure will determine the minimum number of stamps
  of values 1p, 7p, and 10p to make up total postage of  $n$  pence.
▷ Base cases
1  Set up the array  $S$  starting as above (with the
   values  $S[i]$  for  $i = 0, \dots, 10$ ).
2  if  $n \leq 10$  then
3      return  $S[n]$ 
4  else
5      for  $j \leftarrow 11$  to  $n$  do
6           $S[j] = 1 + \min\{S[j-1], S[j-7], S[j-10]\}$ 
7  return  $S[n]$ 
```

As you can see, there is one “for” loop in the body of the pseudo-code, and this dominates the running time of the algorithm. Thus, finding $S[N]$ takes time which is in $O(N)$.

Now we can find $S[n]$ easily as above, but knowing this doesn’t tell us the exact nature of the stamps we need. Right? It would be nice to know that we can make 45p postage with six stamps, *and* that we need one 1p, two 7p, and three 10p stamps to do so. (For some values of n there could certainly be more than one way to do this. For example, we can get 63p with nine 7p stamps, *or* with six 10p and three 1p stamps.)

Well, we can easily do this too with another array called, say, P . Then $P[n]$ will be a vector of length 3 that will tell us what stamps we need to make postage for np . For example, we would have $P[45] = (1, 2, 3)$, meaning that we need one 1p, two 7p, and three 10p stamps. In general, if we have $P[n] = (a, b, c)$, then we take a 1p stamps, b 7p stamps, and c 10p stamps to make np postage. So, for example $P[7] = (0, 1, 0)$, $P[10] = (0, 0, 1)$, and $P[25] = (1, 2, 1)$. (As I said earlier, $P[n]$ isn’t really uniquely defined, but that’s not important for us. We just want some way, using the minimum *number* of stamps, to get n pence.)

The calculation for $S[n]$ doesn’t change. All we do is determine which denomination of postage we use and add it to the appropriate value of $P[n - 1]$, $P[n - 7]$, or $P[n - 10]$. The modified pseudo-code is given below. The addition of vectors in lines 9, 11, and 13 is done component-wise in the usual way, e.g. $(3, 4, 1) + (0, 1, 0) = (3, 5, 1)$. The running time is still $O(n)$.

STAMPS(n)

▷ We assume here that $n \geq 0$.

▷ This procedure will determine the minimum number of stamps of values 1p, 7p, and 10p to make up total postage of n pence. We also return a vector, $P[n]$ which will tell us *how many* of each stamp is necessary.

▷ Base cases

- 1 Set up the array S starting as above (with the values $S[i]$ for $i = 0, \dots, 10$).
- 2 Also set up an array $P[i]$ for $i = 0, \dots, 10$. That is $P[0] = (0, 0, 0)$, $P[1] = (1, 0, 0)$, $P[2] = (2, 0, 0)$, $\dots$, $P[7] = (0, 1, 0)$, etc.

3 **if** $n \leq 10$ **then**

4 return $S[n]$ and $P[n]$

5 **else**

6 **for** $j \leftarrow 11$ **to** n **do**

7 $S[j] = 1 + \min\{S[j-1], S[j-7], S[j-10]\}$

▷ Now set $P[j]$ based on the value of $S[j]$.

In other words, which stamp did we use?

8 **if** $S[j] == (1 + S[j-1])$ **then**

9 $P[j] \leftarrow P[j-1] + (1, 0, 0)$

10 **elseif** $S[j] == (1 + S[j-7])$ **then**

11 $P[j] \leftarrow P[j-7] + (0, 1, 0)$

12 **else**

13 $P[j] \leftarrow P[j-10] + (0, 0, 1)$

14 **return** $S[n]$ and $P[n]$

4. Longest Increasing Subsequence. (This is a classical dynamic programming problem.) Given a sequence of real numbers $A_1, A_2, \dots, A_n$, determine a subsequence (not necessarily contiguous) of maximum length in which the values form a *strictly increasing sequence*.

For example, given the sequence of integers

$$-3, 4, 14, 0, -1, 3, 5, 2, 5, 10$$

the subsequence consisting of 4, 5, 10 is an increasing subsequence, as is $-3, 0, 3, 5, 10$. Your task is to find a longest subsequence so that the values are strictly increasing. (There might be repeated numbers in the original sequence, but there can be no repeats in a longest subsequence since it must be strictly increasing.)

Hint: Define $L(j)$ to be the length of the longest increasing subsequence that ends at (and includes) position j . How can you find $L(j)$ in terms of the values $L(i)$ for $i < j$? Once you know the $L(j)$ values, what's the answer to the original problem? And how long does it take to compute all of the $L(j)$ values?

Solution: As suggested above, we define $L(j)$ to be the length of longest increasing subsequence that ends at (and includes) position j . If we are able to compute these $L(j)$ values, then the solution to our problem is equal to $L = \max\{L(1), L(2), \dots, L(n)\}$. Why? Because the longest

increasing subsequence must end at (and include) one of the positions $1, 2, \dots, n$.

We note that $L(1) = 1$, since the single value A_1 is an “increasing” subsequence. Also, $L(2) = 1$ or 2 , depending upon whether $A_1 \geq A_2$ or $A_1 < A_2$ (recall that the sequence must be strictly increasing).

If we have computed $L(1), L(2), \dots, L(i)$ for some i , how do we use these values to find $L(i+1)$? We compute

$$L(i+1) = 1 + \max\{L(j) : A_j < A_{i+1}\}, \text{ or possibly } L(i+1) = 1 \text{ (see below).}$$

In other words, check all A_j , find those that are less than A_{i+1} . We do this because if $A_j < A_{i+1}$, then we can increase the longest subsequence that ends at A_j by one more value (namely, adding A_{i+1} to that subsequence). Note that if there are no values A_j that are less than A_{i+1} , then we define $L(i+1) = 1$ (as for $L(1)$).

Computing $L(i+1)$ requires us to check all of the values $A_1, \dots, A_i$, i.e. $O(i)$ time. So, in total, computing all of the $L(i)$ values takes $O(1+2+3+\dots+n) = O(n^2)$ time. Then finding the value $L = \max\{L(1), L(2), \dots, L(n)\}$ takes $O(n)$ time. So finding the value L takes $O(n^2)$ time.